

Audit — `qbmizsj/A-GCL` (official repo) vs. Zhang et al., *Med. Image Anal.* 90 (2023) 102932

Clone audited: `github.com/qbmizsj/A-GCL`, HEAD = `08339b7` ("Add files via upload"). Full history checked — 30 commits, two code uploads (`1c117cf`, `08339b7`). Every `.py` compile-checked; the committed `.pyc` files were unmarshalled to recover the author's real (unpublished) source.

Verdict

No. You cannot run this repo unchanged. It does not compile.

`unsupervised/embedding_evaluation.py` has an unterminated function call, and that module is imported by both entry points, so `python agcl_ABIDE.py` dies at import before touching your data.

This is not a version-skew problem you can pin your way out of. The published `.py` is a hand-stripped copy of the author's working file, and the stripping deleted lines mid-statement. The same break exists in **both** uploads (May 2023 and the current HEAD), so it has never been runnable as published.

Fixable in ~20 lines. Patch at the bottom. But budget for the fact that you are reconstructing the author's intent, not running their code.

1. Hard blockers — it will not start

B1. SyntaxError, `unsupervised/embedding_evaluation.py:256`

```
train_score, val_score, ..., fpr, tpr, running_time=
self.embedding_evaluation(encoder,

^
SyntaxError: '(' was never closed
```

The arguments were deleted, the closing paren with them. Python then swallows every following line until it hits `return` inside the open paren. Verified with `py_compile` on both uploads.

Recovered from `__pycache__/embedding_evaluation.cpython-38.pyc` (author's build, `/home/zhang_istbi/.../gcn_project/adgcl_gcn/`), the call was:

```
python

self.embedding_evaluation(encoder, train_loader, valid_loader, test_loader, fla
```

The `.pyc` also proves the published file was **stripped** — the real class has three more methods (`plot_latent_vector`, `plot_latent_vector_3d`, `plot_loss`) that were removed before upload. That is where the damage came from.

B2. `ImportError: cannot import name 'TUEvaluator'`

`agcl_ABIDE.py:13` and `agcl_ABIDE_queue.py:14` do `from datasets import TUEvaluator`. `datasets/__init__.py` is one line: `from .abideDataset import ABIDEDataset`. `TUEvaluator` lives in `datasets/tu_dataset.py:156` and is never re-exported.

B3. `from scipy import interp`

Both entry points, line 19/20. `scipy.interp` is a legacy NumPy alias removed from modern SciPy (fails on the SciPy in any 2024+ environment). **It is never used** — `grep` finds zero call sites. Delete the line.

B4. `UnboundLocalError: running_time`

`embedding_evaluation.py:180-188` only assigns `running_time` when `flag=True`, but returns it unconditionally. The first call — `ee.kf_embedding_evaluation(model.encoder, dataset)` at `agcl_ABIDE.py:72` — uses the default `flag=False`. Crashes on the pre-training evaluation, before epoch 1.

B5. `NameError: test_id`

`embedding_evaluation.py:243` calls `test_id.append(test_index)`. `test_id` is never initialised in the published file. The `.pyc` confirms it was a local in the author's version (alongside `tprs`, `aucs`, `mean_fpr`, all also deleted). Delete the line — nothing reads `test_id`.

B6. Return-arity mismatch

`embedding_evaluation()` as published returns 13 values. `kf_embedding_evaluation` unpacks 15 (`..., fpr, tpr, running_time`). The author's version computed `roc_curve` inside `embedding_evaluation` (`roc_curve` is in the `.pyc`'s name table, `fpr`/`tpr`/`thresholds` in its varnames); the published file has that line stripped but the caller not updated. Fixing B1 alone leaves you with a `ValueError` on unpack.

2. Silent correctness bugs — it runs, and the numbers are wrong

S1. Evaluation throws away your edge weights ⚠️ most important

`unsupervised/encoder/tu_encoder.py:98`

```
python

x, _ = self.forward(batch, x, edge_index, edge_weight)
```

but the signature is

```
python
```

```
def forward(self, batch, x, edge_index, edge_attr=None, edge_weight=None):
```

Your PCC values land in the `edge_attr` slot. `edge_weight` stays `None`. `WGINConv.message` then returns `F.relu(x_j)` with no weighting at all.

Consequence: **training uses the weighted graph; the SVM features used for every reported accuracy come from an unweighted graph**. Every number in Table 2 was produced through this path. It also partly undercuts the Fig. 4 "edge weights help" ablation, since the ablation can only act through the trained encoder weights, never through the evaluation forward pass.

One-word fix: `self.forward(batch, x, edge_index, None, edge_weight)`.

S2. Subject↔feature matching silently pairs the wrong subject

`datasets/abideDataset.py:54-59`

```
python

digit_str = "".join(list(filter(str.isdigit, file))[:-3])
for file_nf in files_nf_ASD:
    if digit_str in file_nf:
        x = ...    # last match wins; no break, no reset, no error
```

`[:-3]` assumes the atlas token contributes **exactly three digits**. Simulated:

ADJ filename	<code>digit_str</code>	unique per subject?
<code>sub_0050002_aal116_all_positive.mat</code>	<code>0050002</code>	✓
<code>sub_0050002_aal166_all_positive.mat</code>	<code>0050002</code>	✓
<code>sub_0050002_shen268_all_positive.mat</code>	<code>0050002</code>	✓
<code>sub_0050002_aal3_all_positive.mat</code> (the README's own example)	<code>00500</code>	✗

With the README's `aal3` naming, `digit_str` collapses to the first five digits and matches essentially every subject. In my simulation over six subjects, **all six graphs received the same node-feature file** — the last one in `os.listdir` order. Node features would carry zero subject-specific information and the model would learn from FC alone.

Second failure mode, independent of naming: if a subject has an ADJ file but no matching NF file, the inner loop never fires and `x` **retains the previous subject's tensor** from the last outer iteration. No exception, no warning.

If you use this loader, name your files `sub_<7-digit-id>_aal116_all_positive.mat` — or, better, replace the whole block with an exact ID regex and a hard `assert`.

S3. Memory-bank loss has the sign inverted (`agcl_ABIDE_queue.py`)

`calc_regloss` (L23-44) returns `-mean_log_prob_pos.mean()` — a standard MoCo-style InfoNCE **loss**, to be *minimised*. The paper (§2.2.2) states the memory-bank term $\mathcal{J}_M$ is **maximised**, i.e. this loss minimised.

```
python
```

```
L201: view_loss = model.calc_loss(x, x_aug) - reg_lambda*reg + cr_lambda*cr_l  
L227: model_loss = model.calc_loss(x, x_aug) - cr_lambda*cr_loss
```

Ascending `view_loss` maximises `cr_loss`. Descending `model_loss` also maximises `cr_loss`. **Both players push the memory-bank term in the same direction, and it is the wrong direction** — away from alignment with the stored negatives' contrast. There is no adversarial structure on this term at all. Under the paper's Eq. (2), `model_loss` should read `+ cr_lambda*cr_loss`.

This matters for you specifically: `agcl_ABIDE_queue.py` is the variant that implements the paper's actual final method ($\lambda_2 = 0.4$, bank length 256). `agcl_ABIDE.py` has no memory bank and is *not* the method in Table 2.

S4. `reg_lambda` pushes the opposite way from the paper's description

Code: `reg = mean(1 - batch_aug_edge_weight)` = **mean drop probability**. `view_loss` contains `- reg_lambda * reg` and is ascended $\Rightarrow$ the view learner is rewarded for making `reg` **small** $\Rightarrow$ **larger** `reg_lambda` **drops fewer edges**.

Paper §2.2.2: *"To force more edges to be dropped ... we need a regularization term to facilitate edge dropping ... and we want to minimize R."*

Under either reading of the paper's ω (keep-probability or drop-probability), the released code moves the drop rate the opposite way from the text. This is the same "literal vs. reversed" convention you have been sweeping in your own repo — **the official code is the reversed convention**, and its default $\lambda_1 = 2.0$ is a strong *retention* pressure. That is consistent with what you observed: at $\lambda = 2.0$ the drop ratio collapses toward zero and the augmentation stops being adversarial.

S5. Memory bank initialised to zeros, counted as negatives

`MemoryBank_Q.__init__` allocates `torch.zeros(256, emb_dim)`. `F.normalize` leaves zero rows as zeros, so every unfilled slot contributes `exp(0) = 1` to the InfoNCE denominator. For the first `256/batch_size` steps the loss is dominated by dummy negatives. Also `requires_grad=True` on a leaf that is never in an optimiser and is only written via `.data` — no gradient ever flows; the flag is inert.

S6. The augmented view discards PCC entirely

Original view: `model(..., batch.edge_weight)` → PCC weights. Augmented view:
`model(..., batch_aug_edge_weight)` → the sigmoid gate **alone**, PCC not multiplied in.

So the positive pair differs not only by dropped edges but by the whole weighting scheme. Defensible as a literal reading of the Bernoulli-mask equation (the paper does say **A** is initialised to all-1s and the mask multiplies **A**), but Fig. 1(b) depicts the augmented graph as carrying the same edge weights. Worth stating explicitly in your methods either way. **Your repo already matches the official code here.**

3. Paper ↔ code discrepancies

#	Paper	Released code	Impact
D1	§3.1.3: "5-fold cross-validation", stated throughout and in every table caption	<code>kf_embedding_evaluation(..., folds=10, ...)</code> . Confirmed from the author's <code>.pyc</code> defaults tuple <code>(10, 128, False)</code> — not a typo in the stripped file	Table 2 numbers are 10-fold. 10-fold on the same N gives higher and lower-variance accuracy than 5-fold
D2	§2.1: node features min-max normalised to [0,1] across the 3 channels; edge weights divided by max abs value	Neither exists anywhere in the Python. Must have been done in the unreleased MATLAB	If your <code>.mat</code> files are unnormalised, you are running a different model. Do this yourself
D3	§2.4: $\tau = 1$	Reparameterisation temperature = 1.0  . But the InfoNCE temperature is hardcoded 0.2 (<code>ginfo_minmax.py:33</code>) and the memory-bank temperature 0.1 (<code>agc1_ABIDE_queue.py:23</code>) — neither appears in the paper	Two undocumented hyperparameters that materially affect the contrastive objective
D4	§3.1.1: AAL1 = 116 ROIs	<code>num_dataset_features=3</code> is hardcoded; node count is inferred from your data, so 90 runs fine	The 80.65% headline is 116 nodes (cerebrum + cerebellum + vermis). AAL-90 is a different atlas configuration and is not what Table 2 measures. If you are comparing to the paper, use 116
D5	§3.1.1: fMRIPrep + confound regression (FD, global signal, mean tissue signals)	README ships a fMRIPrep script but ALFF is computed by DPABI <code>y_alff_fa1ff.m</code> on the AAL3-registered images; the atlas registration script uses AAL3 only	The AAL1/Shen268 preprocessing is not reproducible from what is released

#	Paper	Released code	Impact
D6	Table 2 reports Accuracy / AUC / Precision / Recall / F1	Code computes accuracy, F1, sensitivity, specificity. No AUC. <code>(LinearSVC)</code> has no <code>(predict_proba)</code> ; the author's <code>(.pyc)</code> fed <code>(roc_curve)</code> the <i>hard</i> labels, which makes the AUC column degenerate	The AUC column cannot be reproduced from this code
D7	—	<code>(ASD → y=0), (HC → y=1)</code> <code>((abideDataset.py:77, 114)).</code> <code>(sensitivity())</code> uses <code>(CM[1,1]/(CM[1,1]+CM[1,0]))</code> and <code>(f1_score)</code> defaults to <code>(pos_label=1)</code>	Reported "sensitivity" and "F1" are w.r.t. the control class, not the patient class. Check this against how you report yours
D8	—	<code>(--template)</code> is parsed and never referenced . <code>(num_nodes)</code> computed at <code>(abideDataset.py:74, 111)</code> and never used	Dead args; <code>(--template 116)</code> does nothing

4. Why 80.65% will not reproduce as an honest number

Three mechanisms compound, all in `(agc1_ABIDE.py)`:

1. **Epoch selection on the test fold.** `(best_test_epoch = np.argmax(np.array(test_curve)))` (L254), and separately `(best_f1_test_epoch)`, `(best_sen_test_epoch)`, `(best_spe_test_epoch)` (L258-266). `(BestTestScore)` is therefore the **max over epochs, taken independently per metric**. Accuracy, F1, sensitivity and specificity in a single reported row can come from four different epochs.
2. **A fresh CV split at every evaluation.** `(KFold(n_splits=folds, shuffle=True, random_state=None))` (L241) and `(train_test_split(..., random_state=None))` (L246). With `(eval_interval=5)` over 100 epochs that is 20 independent 10-fold resamplings, and step 1 takes the maximum over all of them.
3. **The encoder is self-supervised on the full dataset**, test folds included, before any split. Standard practice for the InfoGraph/AD-GCL evaluation protocol and not label leakage — but transductive, and it must be stated.

Your nested-CV harness already fixes all three. Expect the honest number to sit meaningfully below 80.65%, and expect that gap to be the *protocol*, not your implementation.

5. Data contract — what your `(.mat)` files must satisfy

Checked against `(abideDataset.py)`:

```
<root>/raw/ASD_ADJ/  sub_0050002_aal116_all_positive.mat  key:
'corr_each_sub'      [R, R] float
<root>/raw/ASD_NF/    alff_sub-0050002.mat                                key:
'alff_value_cache'   [R, 3] float
<root>/raw/HC_ADJ/    ...
<root>/raw/HC_NF/     ...
```

- **Exact key names.** `corr_each_sub` and `alff_value_cache`. Anything else → `KeyError`.
- `'positive'` must be in the ADJ filename — `abideDataset.py:52, 93` skip every file without it.
- **Atlas token must contribute exactly 3 digits** (see S2). `aal116` / `aal166` / `shen268` ✓, `aal3` ✗.
- **Save as MATLAB v5, not `-v7.3`.** `scipy.io.loadmat` cannot read HDF5-backed v7.3 files.
- **Zero the diagonal** of the PCC matrix. `edge_index.nonzero()` (L73, L110) keeps every non-zero entry, so a diagonal of 1s becomes 90 self-loop edges — on top of the $(1 + \text{eps}) * x_r$ root term `WGINConv` already adds. The paper does not specify this; you should.
- **Column order of node features must be `[slow-5, slow-4, classical]`** consistently across all subjects — nothing checks it.
- The `all_positive` in the filename implies negative correlations were zeroed before saving. **This is not described anywhere in the paper.** Decide, then state it in your methods.
- ⚠️ `raw/` is scanned but `processed/data.pt` is cached and **never invalidated**. If you change your `.mat` files, delete `processed/` or you will silently train on the old graphs.

6. Minimal patch

Six edits. Applies to both entry points.

`datasets/__init__.py`

```
python

from .abideDataset import ABIDEDataset
from .tu_dataset import TUEvaluator          # + add
```

`agc1_ABIDE.py` L19 and `agc1_ABIDE_queue.py` L20 — delete:

```
python

from scipy import interp                    # - remove (unused, breaks on mod
```


`unsupervised/encoder/tu_encoder.py` L98 — the edge-weight bug:

```
python

x, _ = self.forward(batch, x, edge_index, None, edge_weight) # was: (batch, x
```

`unsupervised/embedding_evaluation.py` L185-188 — always time:

```
python

val_end = time.time()
running_time = val_end - val_start
if flag:
    print('validation time cost : %.5f sec' % running_time)
```

(and move `val_start = time.time()` out of its `if flag:` at L180-181)

`unsupervised/embedding_evaluation.py` L243 — delete `test_id.append(test_index)`.

`unsupervised/embedding_evaluation.py` L256 — restore the call and match the 13-value return:

```
python

(train_score, val_score, test_score,
 train_f1, val_f1, test_f1,
 train_sen, val_sen, test_sen,
 train_spe, val_spe, test_spe,
 running_time) = self.embedding_evaluation(
    encoder, train_loader, valid_loader, test_loader, flag)
```

Then, before you trust any output:

- `--folds 5` if you are comparing against the paper's stated protocol (default is 10 — D1).
- If you run `agcl_ABIDE_queue.py`, flip S3: `model_loss = model.calc_loss(x, x_aug) + args.cr_lambda * cr_loss`. Run it both ways and report which you used.
- Environment: pin `torch-geometric==2.0.4`. `from torch_geometric.data import DataLoader` was removed in PyG ≥ 2.4 . On `torch>=2.6`, `torch.load` defaults to `weights_only=True` and will fail on `data.pt` — pass `weights_only=False` at `abideDataset.py:22`. `torch-scatter` must be built against your exact torch version.

7. Bottom line

- **Reproduce as-is: no.** B1-B6 stop it at import.

- **Reproduce after the patch: yes, mechanically** — it will train and produce numbers on 90 or 116 ROIs.
- **Reproduce Table 2's 80.65%: not honestly.** D1 (10-fold, not 5) plus the three selection mechanisms in §4 account for a large part of that figure. Your own nested-CV number is the comparable one.
- **Most valuable single line in here:** S1. Fix `tu_encoder.py:98` in your own repo too if you inherited that call — check it, because every accuracy you have measured flows through `get_embeddings`.
- **S3 and S4 are the sign conventions you have been sweeping.** The official code is `reversed` for the drop regulariser and inverted for the memory-bank term relative to the paper text. That settles what "matching the official implementation" means for those two flags — it is not the same as matching the paper.